

Report (Assignment 4)

Quantization From Scratch

Results:

```
Baseline - Memory: 486.7002410888672 MB, Latency: 0.0075s, Perplexity: 49.2670
Whole-Model Quantization - Memory: 486.7002410888672 MB, Latency: 0.0076s, Perplexity: 60.3966
Selective Quantization - Memory: 486.7002410888672 MB, Latency: 0.0076s, Perplexity: 50.3291
```

After applying quantization uniformly across the whole model, perplexity increases significantly from 49.27 (baseline) to 60.40. It implies that certain layers or parameters are more sensitive to quantization, which may decrease model performance. Perplexity increases slightly to 50.33 in the case of selective quantization which indicates that selectively applying quantization to certain layers such as attention and ffn layers of the model helps preserve performance, minimizing the impact on model quality.

Both whole-model and selective quantization exhibit a negligible increase in latency which indicates that the model's numerical precision impacts performance more than runtime.

Both approaches maintain the same memory footprint (486.7 MB) as the baseline. This is because models initialized in Pytorch store their parameters in floating-point precision (torch.float32 or torch.float16) by default, even after quantization.

Bitsandbytes Integration and NF4 Quantization

Results:

```
Evaluating 8-bit Quantization:
Memory Usage (MB): 168.35
Inference Latency (s): 0.0552
Perplexity: 50.78
```

```
Evaluating 4-bit Quantization:
Memory Usage (MB): 127.85
Inference Latency (s): 0.0222
Perplexity: 57.71
```

```
Evaluating NF4 Quantization:
Memory Usage (MB): 127.85
Inference Latency (s): 0.0222
Perplexity: 52.91
```

NF4 (Normalized Float 4-bit) quantization is a specialized quantization method designed to preserve the information content of high-precision floating-point tensors while reducing the bit-width. The input tensor is normalized, effectively

dividing it by its standard deviation. This normalization ensures that values are distributed within a more compact and consistent range. NF4 applies nonlinear scales, mapping values to a discrete set of levels. Unlike linear quantization, NF4 uses levels that are distributed logarithmically or designed to better capture the statistical properties of the data.

Linear quantization maps floating-point values directly to evenly spaced levels between a fixed minimum and maximum range. Key characteristics of linear quantization are:

- Uniform step size between quantized levels.
- Simpler implementation and faster computations.
- Less capable of representing extreme or subtle variations in data.

Linear Quantization is effective for datasets or layers with uniformly distributed values. But it fails to handle distributions with significant skewness, leading to a loss in representational fidelity and reduced accuracy. NF4 Quantization, by aligning better with the distribution of weights and activations, it retains more information and results in less performance degradation. This explains why NF4 quantization shows better perplexity (52.91) compared to linear quantization (57.71) at the same 4 bit-width.

NF4 Quantization is slightly more computational overhead due to normalization and nonlinear mapping. However, it is more memory-efficient for capturing dynamic ranges effectively, potentially reducing the need for higher bit-widths.

Comparison Table

Quantization Method	Memory Usage (MB)	Latency (s)	Perplexity
Baseline	486.70	0.0075	49.27
Whole-Model Quantization	486.70	0.0076	60.40
Selective Quantization	486.70	0.0076	50.33
Bitsandbytes 8-bit Quantization	168.35	0.0552	50.78
Bitsandbytes 4-bit Quantization	127.85	0.0222	57.71
Bitsandbytes NF4 Quantization	127.85	0.0222	52.91

Lower bit-widths or whole-model quantization introduce more errors in parameter representation, increasing perplexity. Selective quantization and nonlinear methods like NF4 reduce perplexity to some extent. Model quantized using Bitsandbytes is performing better than whole model quantization at the same 8 bit-width.

Due to Bitsandbytes 8-bit quantization, latency increased from 0.0076 to 0.0552s, likely due to the computational overhead of handling reduced precision at runtime. However 4-bit and NF4 quantization causes the latency to decrease to 0.0222s,

showing an improvement in runtime efficiency compared to 8-bit. This reflects the smaller memory footprint and faster memory bandwidth utilization.

In case of whole-model and selective quantization memory usage remained constant at 486.70 MB, indicating no size reduction. In case of Bitsandbytes, 8-bit quantization causes memory usage to drop significantly to 168.35 MB, reflecting a halving of storage requirements compared to full-precision parameters. In case of 4-bit and NF4 quantization, memory usage decreased further to 127.85 MB, representing a 4x reduction in size.

Impact of Quantization Approach on Metrics

Quantization Method	Perplexity	Latency	Memory Usage	Overall Impact
Whole-Model	High increase	Minimal	No reduction	Poor: Accuracy loss without memory or speed gains.
Selective	Slight increase	Minimal	No reduction	Good: Balances accuracy and efficiency well.
8-bit (Bitsandbytes)	Minimal increase	Slight increase	Significant reduction	Excellent: Retains accuracy with large memory savings.
4-bit (Bitsandbytes)	Moderate increase	Significant decrease	Significant reduction	Good for memory and latency but accuracy suffers.
NF4 (Bitsandbytes)	Moderate increase	Significant decrease	Significant reduction	Best balance of accuracy, speed, and memory.